UNIT 4.INTRODUCTION TO OOPS (OBJECT-ORIENTED PROGRAMMING)

Definition:
Object-Oriented Programming (OOP) is a programming paradigm that uses **objects and classes** to design and develop programs.

Basic Concepts of OOP

- **Class** → Blueprint of an object
- **Object** → Instance of a class
- **Encapsulation** → Binding data and methods
- **Inheritance** → Reusing properties of another class
- **Polymorphism** → Same function, different behavior

Key Concepts in OOP

1. Class

A class defines properties (variables) and behaviors (methods).

```
class Student:
    name = "John"
```

---

2. Object

An object is an instance of a class.

```
s1 = Student()
print(s1.name)
```

---

3. Encapsulation

Combining data and functions into one unit.

---

4. Inheritance

A class can inherit properties from another class.

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    pass
```

---

5. Polymorphism

Same function behaves differently.

```
def add(a, b):
    return a + b
```

Introduction to Object-Oriented Programming (OOP)

Introduction

Object-Oriented Programming (OOP) is a programming approach that organizes software design around objects and classes rather than functions and logic. It helps in building modular, reusable, and maintainable code.

Definition

OOP is a paradigm that uses objects, which contain data and methods, to represent real-world entities.

Basic Concepts of OOP

1. Class

A class is a blueprint for creating objects.

```
class Person:
    def __init__(self, name):
        self.name = name
```

---

2. Object

An object is an instance of a class.

```
p1 = Person("Alice")
print(p1.name)
```

---

3. Encapsulation

Binding data and methods together.

✓ Provides data hiding
✓ Improves security

---

4. Inheritance

Allows one class to acquire properties of another.

```
class Parent:
    def show(self):
        print("Parent")

class Child(Parent):
    pass
```

✓ Promotes code reuse

---

5. Polymorphism

Same method name with different behavior.

✓ Example: function overloading concept

---

6. Abstraction

Hides implementation details and shows only essential features.

Advantages of OOP

- Code reusability
- Easy maintenance
- Better data security
- Modular programming

✅ OOP CONCEPTS IN PYTHON

Introduction

Object-Oriented Programming (OOP) in Python is a programming paradigm that uses **classes and objects** to structure programs. Python supports OOP to create modular, reusable, and efficient code.

Definition

OOP in Python is a method of programming that organizes data and functions into objects, which interact with each other.

◆ Core OOP Concepts in Python

---

1. Class

A class is a blueprint for creating objects.

```
class Student:
    def __init__(self, name):
        self.name = name
```

---

2. Object

An object is an instance of a class.

```
s1 = Student("John")
print(s1.name)
```

---

3. Constructor (init)

A special method used to initialize objects.

```
class Person:
    def __init__(self, age):
        self.age = age
```

---

4. Encapsulation

Binding data and methods into a single unit.

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance   # private variable

    def show(self):
        print(self.__balance)
```

✓ Provides data hiding
✓ Improves security

---

5. Inheritance

Allows a class to inherit properties from another class.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    pass
```

✓ Promotes code reuse

---

6. Polymorphism

Same method behaves differently.

```
def add(a, b):
    return a + b
```

✓ Works with different data types

---

7. Abstraction

Hides internal implementation and shows only necessary details.

✓ Achieved using classes and methods

◆ Types of Inheritance in Python

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance

◆ Advantages of OOP in Python

- Code reusability
- Easy maintenance
- Better organization
- Data security

✅ INHERITANCE AND NAMESPACE IN PYTHON

Inheritance

Inheritance is an OOP concept where one class (child) acquires properties and methods of another class (parent).

Namespace

A namespace is a container that holds names (variables, functions) and their corresponding values.

Types of Inheritance in Python

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

Namespace Types

- Local Namespace
- Global Namespace
- Built-in Namespace

Inheritance in Detail

Example: Single Inheritance

```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.show()
```

Multiple Inheritance

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

Namespace in Detail

Example

```
x = 10   # Global namespace

def func():
    y = 20   # Local namespace
    print(x, y)

func()
```

Inheritance and Namespace in Python

Introduction

Inheritance and namespace are important concepts in Python programming. Inheritance supports code reuse, while namespace helps organize and manage variable names efficiently.

◆ 1. Inheritance

Definition

Inheritance is a mechanism in which a class acquires properties and methods of another class.

Syntax

```
class Parent:
    pass

class Child(Parent):
    pass
```

Types of Inheritance

1. Single Inheritance

One child inherits from one parent.

2. Multiple Inheritance

A child inherits from multiple parents.

3. Multilevel Inheritance

Inheritance in multiple levels.

4. Hierarchical Inheritance

Multiple child classes inherit from one parent.

Example

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.sound()
d.bark()
```

Advantages

- Code reusability
- Reduces redundancy
- Easy maintenance

◆ 2. Namespace

Definition

A namespace is a system that ensures all names in a program are unique and can be used without conflict.

Types of Namespace

1. Local Namespace

Inside a function.

2. Global Namespace

Defined at the top level of a program.

3. Built-in Namespace

Predefined names in Python (e.g., `len()`, `print()`).

Example

```
x = 100   # Global

def display():
    y = 50   # Local
    print(x, y)

display()
```

Scope Resolution (LEGB Rule)

Python resolves names in the following order:

1. Local
2. Enclosing
3. Global
4. Built-in

Advantages of Namespace

- Avoids naming conflicts
- Organizes code
- Improves readability

✅ BASICS OF NUMPY, SCIPY AND PANDAS

NumPy

NumPy is a Python library used for numerical computations and working with arrays.

SciPy

SciPy is a scientific computing library built on NumPy, used for advanced mathematical operations.

Pandas

Pandas is a Python library used for data analysis and manipulation using data structures like Series and DataFrame.

Features

NumPy

- Supports multi-dimensional arrays
- Fast numerical operations
- Mathematical functions

SciPy

- Built on NumPy
- Provides scientific tools
- Used in optimization, integration

Pandas

- Data analysis tool
- Handles structured data
- Provides DataFrame and Series

NumPy Basics

```
import numpy as np

arr = np.array([1, 2, 3])
print(arr)
```

Operations:

```
arr + 2
arr * 3
```

SciPy Basics

```
from scipy import stats

data = [1, 2, 3, 4]
print(stats.mean(data))
```

Pandas Basics

```
import pandas as pd

data = {"name": ["A", "B"], "age": [20, 25]}
df = pd.DataFrame(data)
print(df)
```

Basics of NumPy, SciPy, and Pandas

Introduction

NumPy, SciPy, and Pandas are popular Python libraries used in data analysis, scientific computing, and data manipulation. They simplify complex operations and improve performance.

◆ 1. NumPy

Definition

NumPy (Numerical Python) is a library used for working with arrays and numerical data.

Features

- Multi-dimensional arrays (ndarray)
- Fast computations
- Mathematical functions

Example

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
print(arr * 2)
```

◆ 2. SciPy

Definition

SciPy is a library built on NumPy used for scientific and technical computing.

Features

- Optimization
- Statistics
- Integration
- Signal processing

Example

```
from scipy import stats

data = [10, 20, 30]
print(stats.mean(data))
```

◆ 3. Pandas

Definition

Pandas is a library used for data manipulation and analysis.

Features

- DataFrame and Series
- Data cleaning and filtering
- Handling missing data

Example

```
import pandas as pd

data = {"Name": ["John", "Anna"], "Age": [22, 24]}
df = pd.DataFrame(data)
print(df)
```

◆ Advantages

- Efficient data handling
- Fast computations
- Easy data analysis
- Widely used in real-world applications